



PRÁCTICA_13: PROYECTO NIVEL INTERMEDIO

Vamos a diseñar una web para una empresa, es una web para un negocio en la que mostraremos la información de la misma, el catálogo de servicios....

A partir del fronted que se te proporciona tendrás que realizar las tareas que se te indican:

1.Determinar cuantas app vamos a crear según organicemos el proyecto. Diferenciamos entre páginas estáticas y dinámicas, para estas últimas el cliente manejará el panel de administración

- Páginas estáticas: portada, historia, visítanos
- Páginas dinámicas: servicios, blog, contacto



La estructura de servicios siempre es la misma, Un título un subtítulo, una imagen de fondo y una descripción.

En el blog tenemos las entradas, los post y también tienen siempre una estructura que se va repitiendo: una fecha de publicación, un título, una imagen...

Vamos a crear dos app, una para cada parte: **servicios**, **blog**

Por otro lado tenemos elementos que debemos automatizar, el pie de página con los iconos a las redes sociales. Lo gestionaremos en una app llamada **social**

El enlace a la página genérica de política de privacidad. Para esto creamos otra app llamada **paginas**

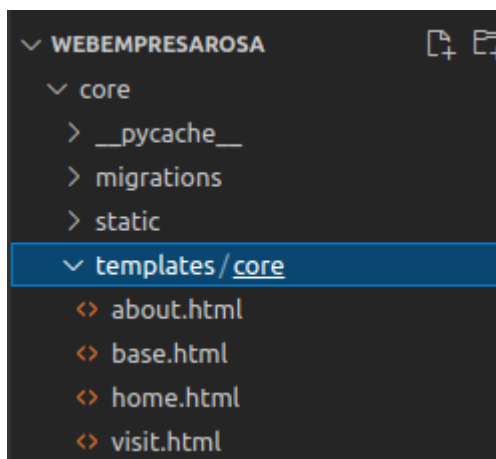
La parte contacto será a su vez otra app que llamaremos **contacto**

La parte estática se suele incluir una app a la que se le suele llamar **core**(núcleo)

De momento, crearemos esas 6 aplicaciones en nuestro proyectos

Pasos a seguir:

- Crea la **app core** con sus correspondientes plantillas : base, home, about, visit
- Añadimos las vista y dejamos para el siguiente apartado las url.



```
from django.shortcuts import render

# Create your views here.
def home(request):
    render(request, 'home.html')

def about(request):
    render(request, 'about.html')

def visit(request):
    render(request, 'visit.html')
```

Si trabajamos como hasta ahora, vamos a tener muchos path definidos en urls.py. En aplicaciones muy grandes puede ser impracticable porque tendríamos muchos imports a las vistas

2. Organizamos nuestras urls

Vamos a crear urls para nuestras app y luego las importamos desde el urls del proyecto. Así lo hace django con el administración

a) Creamos un archivo llamado urls.py en la app core

```
from django.urls import path

urlpatterns = []
```

b) Importamos nuestro archivo de vistas

```
from . import views
```

c) Añadimos las rutas

```
urlpatterns = [
    path('', views.home, name="home"),
    path('about', views.about, name="about"),
    path('visit', views.visit, name="visit")
]
```

d) En el archivo urls del proyecto tenemos que importar los path del core. Nos dice en ese mismo archivo como hacerlo

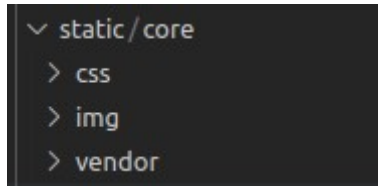
```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    #path('core/', include('core.urls')), si pongo ésto, en las url tendría que poner
    #localhost:8000/core/about
    #No queremos ésto, queremos localhost:8000/about por tanto
    path('', include('core.urls'))
]
```

3. Los templates

Tienes que identificar todas las partes comunes de nuestras plantillas y crear una estructura de herencia utilizando archivos estáticos

a) Los archivos estáticos los colocarás en una estructura de carpetas como la que ves en la imagen

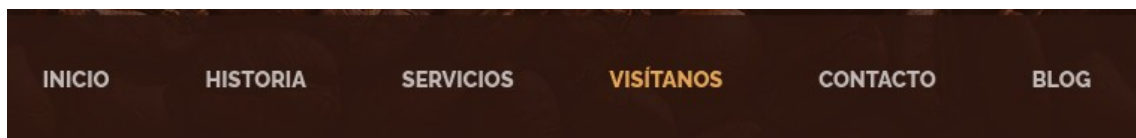


Dentro, obviamente, de la app core

b) Debes actualizar las url en el menú

4. Resaltar la sección seleccionada en el menú

Cada vez que pulsamos una opción de menú queremos que se quede de otro color



Para ello usaremos:

- La propiedad path del objeto request, que nos da la url que estamos visitando
- La propiedad active de css que resalta una opción de menú

```
<li class="nav-item px-lg-4 {% if request.path == '/' %}active{% endif %}">
  <a class="nav-link text-uppercase text-expanded" href="{% url 'home' %}">Inicio</a>
</li>
<li class="nav-item px-lg-4 {% if request.path == '/about/' %}active{% endif %}">
  <a class="nav-link text-uppercase text-expanded" href="{% url 'about' %}">Historia</a>
</li>
```

Ya tenemos funcionando correctamente nuestro core, pasemos al resto de app

5. App Service

Hay que hacer lo mismo que en la app portfolio de la práctica anterior

a) Lo primero configurar los archivos media

```
MEDIA_URL='/media/'
MEDIA_ROOT=os.path.join(BASE_DIR, 'media')
```

b) Crea la nueva app

c) Crea el modelo con la siguiente estructura

- **Title:** Un título con 200 caracteres de longitud máxima.
- **Subtitle:** Un subtítulo con 200 caracteres de longitud máxima.
- **Content:** Un texto de tamaño indefinido.
- **Image:** Una imagen para mostrar de fondo almacenada en el directorio *services* (dentro de media).
- **Created:** Un campo automático para gestionar la fecha y hora de creación.
- **Updated:** Un campo automático para gestionar la fecha y hora de última actualización.

```
services > models.py > Service > image
1 from distutils.command import upload
2 from operator import mod
3 from tabnanny import verbose
4 from turtle import title
5 from django.db import models
6
7 # Create your models here.
8 class Service(models.Model):
9     title=models.CharField(max_length=200, verbose_name="Título")
10    subtitle=models.CharField(max_length=200,verbose_name="Subtítulo")
11    content=models.TextField(verbose_name="Contendio")
12    image=models.ImageField(verbose_name="Imagen",upload_to="services")
13    created=models.DateField(verbose_name="Fecha de creación",auto_now_add=True)
14    updated=models.DateTimeField(verbose_name="Fecha última modificación", auto_now=True)
15
16
17    class Meta:
18        verbose_name='Servicio'
19        ordering=['-created']
20
21    def __str__(self):
22        return self.title
```

No olvides hacer las migraciones

d) Configura la app para que se maneje desde el panel de administración.

Muestra los campos especiales created y updated en modo lectura

```
services > admin.py > ...
1  import imp
2  from django.contrib import admin
3  from .models import Service
4  # Register your models here.
5
6  class ServiceAdmin(admin.ModelAdmin):
7      readonly_fields=['created','updated']
8
9  admin.site.register(Service, ServiceAdmin)
10
```

Recuerda que debes crear un súper usuario para el panel de administración

e)Pongamos el panel de administración en español

En el archivo settings cambiamos el valor de LANGUAGE_CODE a 'es'

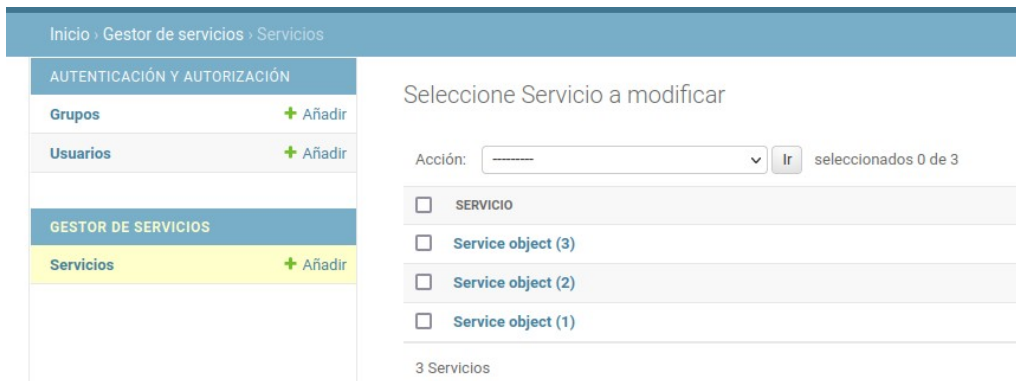
En el archivo apps añadimos la propiedad verbose_name='Texto que queramos mostrar'

```
class ServicesConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'services'
    verbose_name='Gestor de servicios'
```

En el archivo settings, en las apps instaladas

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'core',
    'services.apps.ServicesConfig'
]
```


f) Abre el panel de administrador y añade los tres servicios



Crea la carpeta templates y configura las url para que funcione esta opción

```
{% extends 'base.html' %} {% load static %} {% block title %}Visítanos{% endblock %} {% block content %} {% for s in servicios %}

<section class="page-section">
  <div class="container">
    <div class="product-item">
      <div class="product-item-title d-flex">
        <div class="bg-faded p-5 d-flex mr-auto rounded">
          <h2 class="section-heading mb-0">
            <span class="section-heading-upper">{{s.subtitle}}</span>
            <span class="section-heading-lower">{{s.title}}</span>
          </h2>
        </div>
      </div>
      
      <div class="product-item-description d-flex ml-auto">
        <div class="bg-faded p-5 rounded">
          <p class="mb-0">{{s.content}}</p>
        </div>
      </div>
    </div>
  </div>
</section>

{% endfor %} {% endblock %}
```

6. App Blog

Esta app la utilizaremos para relacionar modelos, a través de claves foráneas o de relaciones muchos a muchos

a) Creamos la app y definimos los modelos

- Categoría: Contiene nombre y fecha de creación y modificación
Clase meta para que nos aparezcan los títulos en español
en el panel y se ordenen por fecha de creación
Método str

- Post: título, contenido, fecha de publicación (no es automática, la decide quien escribe el post), imagen, autor, lista de categorías (de las definidas en el modelo de arriba), fecha de creación y actualización (para nosotros)

La fecha de publicación por defecto tendrá valor, para dárselo utilizamos la librería `timezone` de `django.utils`

`timezone.now()`

La imagen es opcional por tanto debe llevar los parámetros `null` y `blank` a `True`. Si la hay, se sube a blog

A continuación definimos los dos campos que estarán “relacionados” con otros modelos:

autor: debe ser un usuario de los que tendremos almacenados y gestionados con nuestro panel de administración, por tanto debemos importar el modelo `User` que utilizaremos en el panel.

```
from django.contrib.auth.models import User
```

Este campo lo definimos como una clave foránea **`models.ForeignKey`**, el primer campo debe ser el modelo con el que enlazamos, además vamos a definir el tipo de relación, en nuestro caso `on_delete=models.CASCADE` para que no se nos queden entradas en el blog si hemos eliminado al autor. Pero ésto es opcional, hay más posibilidades como ya sabéis

categorías: tenemos que relacionar este campo con el modelo definido antes, pero en este caso tenemos que poder elegir varias categorías por tanto la relación campo - modelo es diferente del caso anterior **`models.ManyToManyField`** dónde el primer campo será el modelo a relacionar

Sólo nos queda añadir en settings la app y hacer las migraciones.

b) Creamos el administrador para esta app

```
blog > admin.py > ...
1  from django.contrib import admin
2  from .models import Category, Post
3
4  # Register your models here.
5  class CategoryAdmin(admin.ModelAdmin):
6      readonly_fields=('created', 'updated')
7
8
9  class PostAdmin(admin.ModelAdmin):
10     readonly_fields=('created', 'updated')
11
12     admin.site.register(Category, CategoryAdmin)
13     admin.site.register(Post, PostAdmin)
```

c) Corremos la aplicación, cargamos el panel de administración y vamos a meter las entradas que tenemos en el front

7. App Social (usuario pink , clave pink)

Permitirá a los clientes configurar sus redes sociales.

a) Creamos la app

b) Implementamos el modelo

```
from django.db import models

# Create your models here.
class Link(models.Model):
    key=models.SlugField(verbose_name="Nombre clave", max_length=100, unique=True)
    name=models.CharField(verbose_name="Red social", max_length=200)
    url=models.URLField(verbose_name="Enlace", max_length=200, null=True, blank=True)
    created=models.DateField(verbose_name="Fecha de creación", auto_now_add=True)
    updated=models.DateTimeField(verbose_name="Fecha última modificación", auto_now=True)

    class Meta:
        verbose_name='Enlace'
        ordering=['name']

    def __str__(self):
        return self.name
```

El tipo de campo usado en atributo key es una forma de generar una URL válida, generalmente utilizando datos ya obtenidos. Por ejemplo, usar el título de un artículo para generar una URL.

Supongamos que nuestro blog tiene una publicación con el título 'El libro de Django de Geeksforgeeks' con la clave principal id = 2. Podríamos referirnos a esta publicación con

www.geeksforgeeks.org/posts/2

O podemos hacer referencia al título como

www.geeksforgeeks.org/posts/The Django book by Geeksforgeeks.

Pero el problema es que los espacios no son válidos en las URL, deben reemplazarse por % 20, lo cual es feo y no resuelve una URL significativa. Otra opción puede ser

www.geeksforgeeks.org/posts/the-django-book-by-geeksforgeeks

Todas las letras se reducen y los espacios se reemplazan por guiones -

Obviamente debe ser único

- c) Añadimos la app en settings
- d) Generamos migraciones
- e) En SocialConfig añadimos propiedad verbose_name
- f) Generamos el administrador

```
from django.contrib import admin
from .models import Link
# Register your models here.

class linkAdmin(admin.ModelAdmin):
    readonly_fields = ('created', 'updated')

admin.site.register(Link, linkAdmin)
```

Abrimos el panel de administración y añadimos algunos enlaces

Modificar Enlace

Twitter

Nombre clave:

Red social:

Enlace: Actualmente: <http://twitter.com>
Cambiar:

Fecha de creación: 19 de Abril de 2022

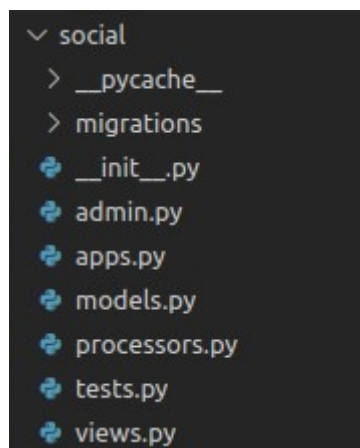
Fecha última modificación: 19 de Abril de 2022 a las 08:41

Añadimos facebook e instagram, ésta última sin el enlace a su web

Sólo nos queda asociar estos enlaces con los iconos de la página. Queremos hacerlo sólo una vez y que nos sirva en todos los templates: **Procesador de contexto**

El contexto es un diccionario que manejan los templates sin necesidad de pasarlo en las vistas. Lo que vamos a hacer es “extender” ese diccionario, es decir añadir datos nuestros.

Para hacer ésto, necesitamos crear una archivo llamado processors.py en al app



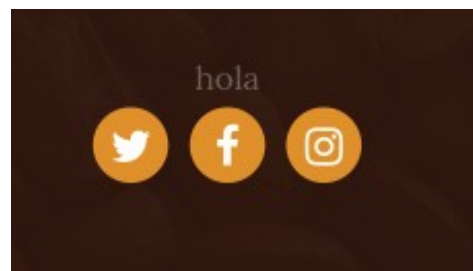
Dentro implementamos la función en la que definiremos nuestro diccionario que amplía el suyo, veamos un ejemplo

```
def ctx_dict(request):  
    ctx={'test':'hola'}  
    return ctx
```

Si añadimos en settings esta ampliación de contexto, podremos usar en cualquier template de cualquier app la variable test

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': ['core/templates/core', 'services/templates/services'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
                'social.processors.ctx_dict'  
            ],  
        },  
    },  
]
```

```
<!-- Pié de página -->  
<footer class="footer text-faded"  
    <div class="container ">  
        {{test}}  
    <p class="m-0 ">
```



Implementa la función `ctx_dict` para que recorra los links que hemos definido y genere el correspondiente diccionario, dónde las claves serán los campos key definidos y los valores los campos url definidos también en el modelo Link

```

from .models import Link

def ctx_dict(request):
    ctx={}
    links=Link.objects.all()
    for l in links:
        ctx[l.key]=l.url
    return ctx

```

El template base quedaría

```

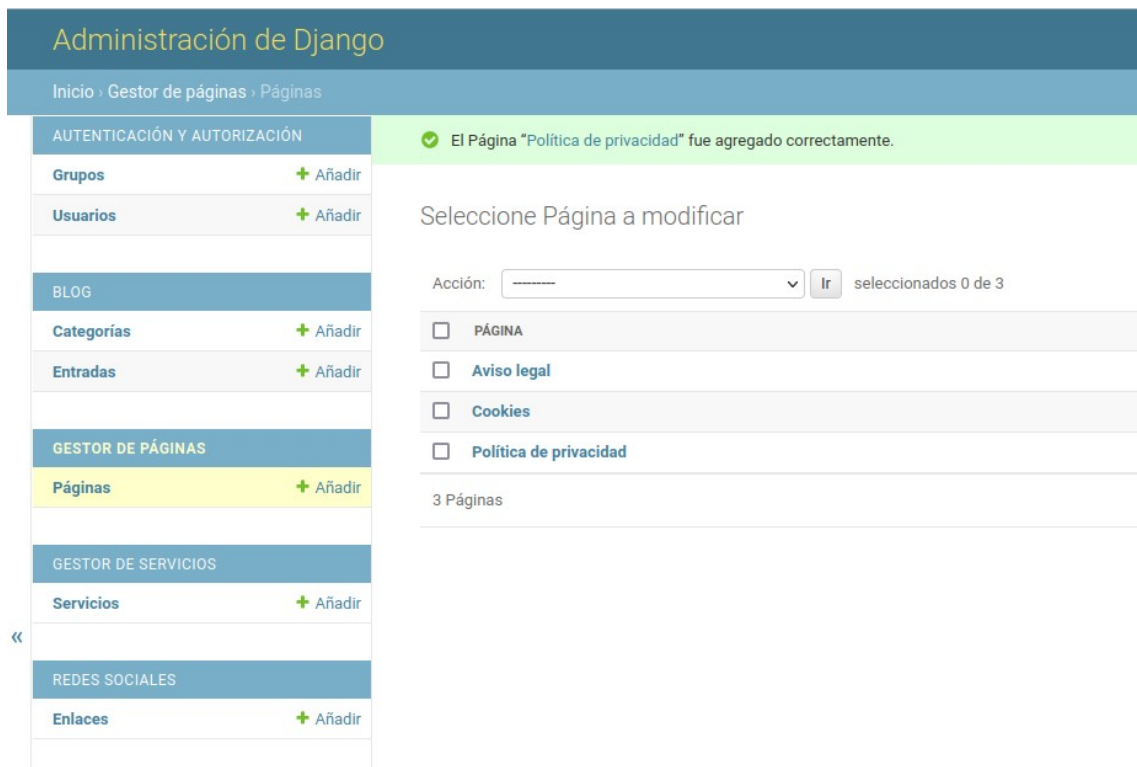
<!-- Pié de página -->
<footer class="footer text-faded text-center py-5 ">
    <div class="container ">
        <p class="m-0 ">
            {% if LINK_TWITTER %}
            <a href="{{LINK_TWITTER}}" class="link ">
                <span class="fa-stack fa-lg ">
                    <i class="fa fa-circle fa-stack-2x "></i>
                    <i class="fa fa-twitter fa-stack-1x fa-inverse "></i>
                </span>
            </a>
            {% endif %} {% if LINK_FACEBOOK %}
            <a href="{{LINK_FACEBOOK}}" class="link ">
                <span class="fa-stack fa-lg ">
                    <i class="fa fa-circle fa-stack-2x "></i>
                    <i class="fa fa-facebook fa-stack-1x fa-inverse "></i>
                </span>
            </a>
            {% endif %} {% if LINK_INSTAGRAM %}
            <a href="{{LINK_INSTAGRAM}}" class="link ">
                <span class="fa-stack fa-lg ">
                    <i class="fa fa-circle fa-stack-2x "></i>
                    <i class="fa fa-instagram fa-stack-1x fa-inverse "></i>
                </span>
            </a>
            {% endif %}
        </p>
    </div>
</footer>

```

8. App Páginas de contenido secundario (políticas de privacidad....)

- a) Creamos la app
- b) Implementamos el modelo : título, contenido, creación y modificación, ordenados por título
- c) Añadimos la app en settings
- e) En PageConfig añadimos propiedad verbose_name
- f) Generamos el administrador
- g) Hacemos las migraciones

Desde el panel de administrador generamos tres páginas



Al hacer click en los enlaces del pie de página debemos cargar estas páginas:

- Añadir la vista correspondiente que retorna una página por su id_page
- Configurar urls.py
- Adaptar la página sample.html que se te proporciona
- Utiliza el mecanismo anterior de extender el contexto para cargar las páginas

Al ejecutar la aplicación veremos que nos queda algo así



El criterio de ordenación lo hemos puesto asociado al título. Vamos a cambiarlo para decidir un orden numérico porque queremos que nos aparezca Política de privacidad, Cookies, Aviso legal. Al haberlas creado a la vez no podemos usar este criterio que en principio sería una opción.

Añadimos un campo al modelo de tipo `SmallInteger`, con valor por defecto = 0 y cambiamos el campo de ordenación

```
# Create your models here.
class Page(models.Model):

    title=models.CharField(verbose_name="Título",max_length=200)
    content= models.TextField(verbose_name="Contenido")
    order=models.SmallIntegerField(verbose_name="Orden",default=0)
    created=models.DateTimeField(verbose_name="Fecha de creación",auto_now_add=True)
    updated=models.DateTimeField(verbose_name="Fecha última modificación", auto_now=True)

    class Meta:
        verbose_name='Página'
        ordering=['order']

    def __str__(self):
        return self.title
```

Para que se nos muestre este campo nuevo en el panel de administración


```
from django.contrib import admin
from .models import Page
# Register your models here.
class PageAdmin(admin.ModelAdmin):
    readonly_fields=('created', 'updated')
    list_display=('title', 'order')

admin.site.register(Page, PageAdmin)
```

Podemos entrar en el panel y cambiar los valores de este campo para que las páginas se ordenen como queremos.

9. App Contacto (Formularios)

Crea la app, genera la vista y las urls para que se muestre el template contact.html (debes adaptarla del fronted)

Una vez que tienes funcionando la app debes generar el formulario contacto para automatizar este proceso.